

Object to Primitive Conversion - Simple Hinglish Summary

Source: <https://javascript.info/object-to-primitive>

Main baat

Jab objects ko add ((obj1 + obj2)), subtract ((obj1 - obj2)) ya (alert(obj)) se print karte hain to kya hota hai?

JavaScript me operators ko objects ke liye **customize nahi kar sakte** (Ruby ya C++ jaisi languages me kar sakte hain). Aise operations me **objects automatically primitives me convert** ho jate hain, phir operation un primitives par hota hai, aur result bhi **primitive** hota hai.

Ye ek important limitation hai: (obj1 + obj2) ka result **dusra object nahi ho sakta**. Yaani vectors ya matrices ke objects banakar unhe add karke "summed" object nahi mil sakta.

Isliye real projects me objects ke saath maths nahi hota. Agar hota hai, to aksar wo **coding mistake** hoti hai.

Is chapter ke do maqsad:

1. Galti se aisa operation ho jaye to samajh sakein ki kya ho raha hai.
2. Kuch exceptions me aisa achha bhi lagta hai, jaise (Date) objects ko subtract ya compare karna.

1. Conversion ke rules

1. **Boolean me conversion nahi hota.** Boolean context me **saare objects (true)** hote hain. Sirf numeric aur string conversion hote hain.
2. **Numeric conversion:** jab objects ko subtract karte hain ya math functions lagate hain. Jaise (Date) objects: (date1 - date2) dono dates ka time difference deta hai.
3. **String conversion:** aam taur par (alert(obj)) jaise output me hota hai.

Ye conversions hum **special object methods** se khud implement kar sakte hain.

2. Hints (kaunsa conversion lagega?)

JS kaise decide karta hai ki kaunsa conversion lagana hai? Iske **teen variants** hain, jinhe **"hints"** kehte hain:

"string" hint

Jab operation string expect karta hai, jaise (alert):

javascript

```
// output
alert(obj);

// object ko property key ki tarah use karna
anotherObj[obj] = 123;
```

"number" hint

Jab maths ho raha ho:

```
javascript

// explicit conversion
let num = Number(obj);

// maths (binary plus ko chhodkar)
let n = +obj; // unary plus
let delta = date1 - date2;

// less/greater comparison
let greater = user1 > user2;
```

Zyadatar built-in math functions me bhi aisa conversion hota hai.

"default" hint

Kabhi kabhi hota hai, jab operator ko pakka nahi pata ki kaunsa type chahiye.

Jaise binary plus (+) strings ke saath (jodta hai) aur numbers ke saath (add karta hai) dono kaam kar sakta hai. Isliye object milne par ye "default" hint use karta hai.

Ye tab bhi hota hai jab object ko (==) se string, number ya symbol se compare karte hain.

```
javascript

// binary plus "default" hint use karta hai
let total = obj1 + obj2;

// obj == number "default" hint use karta hai
if (user == 1) { ... };
```

Dhyan: (<) (>) comparison operators strings aur numbers dono ke saath kaam kar sakte hain, phir bhi wo "number" hint use karte hain, "default" nahi. Ye historical reasons se hai.

Practical baat: Saare built-in objects (ek case chhodkar: Date) "default" ko "number" ki tarah hi implement karte hain. Hume bhi aisa hi karna chahiye.

Conversion ke liye JS 3 methods dhundhta hai

1. Agar `obj[Symbol.toPrimitive](hint)` method hai to use call karo.
2. Nahi to, agar hint `"string"` hai:
 - `obj.toString()` ya `obj.valueOf()` try karo, jo bhi ho.
3. Nahi to, agar hint `"number"` ya `"default"` hai:
 - `obj.valueOf()` ya `obj.toString()` try karo, jo bhi ho.

3. `Symbol.toPrimitive`

Ek built-in symbol `Symbol.toPrimitive` hai jise conversion method ke naam me use karte hain:

javascript

```
obj[Symbol.toPrimitive] = function(hint) {  
  // yahan is object ko primitive me convert karne ka code  
  // ye primitive value return karna zaruri hai  
  // hint = "string", "number", "default" me se ek  
};
```

`Symbol.toPrimitive` method ho to wo **saare hints ke liye** use hota hai, aur koi aur method nahi chahiye.

javascript

```
let user = {  
  name: "John",  
  money: 1000,  
  
  [Symbol.toPrimitive](hint) {  
    alert(`hint: ${hint}`);  
    return hint == "string" ? `{name: "${this.name}"}` : this.money;  
  }  
};  
  
// conversions demo:  
alert(user);           // hint: string -> {name: "John"}  
alert(+user);          // hint: number -> 1000  
alert(user + 500);     // hint: default -> 1500
```

`user` conversion ke hisaab se ya to self-descriptive string ya paise ki raashi ban jata hai. Ek hi method saare conversion cases handle karta hai.

4. toString / valueOf

Symbol.toPrimitive na ho to JS toString aur valueOf methods dhundhta hai:

- **"string" hint:** pehle toString call karo. Wo na ho ya object return kare to valueOf call karo. (String conversion me toString ki priority.)
- **Baaki hints:** pehle valueOf call karo. Wo na ho ya object return kare to toString call karo. (Maths me valueOf ki priority.)

Ye methods **purane zamane** ke hain. Ye symbols nahi, "regular" string-naam wale methods hain. Ye conversion implement karne ka **"old-style"** tarika hai.

Ye primitive value return karne chahiye. Agar toString ya valueOf object return kare to wo **ignore** ho jata hai (jaise method hi na ho).

Plain object ke default methods:

- toString → string "[object Object]" return karta hai
- valueOf → object khud ko return karta hai

javascript

```
let user = {name: "John"};

alert(user); // [object Object]
alert(user.valueOf() === user); // true
```

Isliye object ko string ki tarah use karo (jaise alert) to default me [object Object] dikhta hai. Default valueOf object khud ko return karta hai, isliye ignore ho jata hai. Ise maan lo ki wo hai hi nahi.

Customize karne ke liye toString aur valueOf implement karo:

javascript

```

let user = {
  name: "John",
  money: 1000,

  // hint="string" ke liye
  toString() {
    return `${name: "${this.name}"}`;
  },

  // hint="number" ya "default" ke liye
  valueOf() {
    return this.money;
  }
};

alert(user);      // toString -> {name: "John"}
alert(+user);     // valueOf -> 1000
alert(user + 500); // valueOf -> 1500

```

Behavior wahi hai jo `Symbol.toPrimitive` wale example me tha.

Sirf `toString` implement karna: Aksar hume saare primitive conversions ke liye ek "catch-all" jagah chahiye. Tab sirf `toString` implement kar sakte hain:

```

javascript

let user = {
  name: "John",

  toString() {
    return this.name;
  }
};

alert(user);      // toString -> John
alert(user + 500); // toString -> John500

```

`Symbol.toPrimitive` aur `valueOf` na ho to `toString` saare primitive conversions handle karta hai.

Conversion "hinted" type hi return kare, zaruri nahi

Saare primitive-conversion methods ke liye zaruri baat: wo **zaruri nahi ki "hinted" primitive hi return karein.**

Is par koi control nahi ki `toString` exactly string return kare, ya `"number"` hint par `Symbol.toPrimitive` number return kare.

Sirf ek zaruri cheez: ye methods **primitive** return karein, **object nahi**.

Historical note: `toString` ya `valueOf` object return kare to error nahi aata, bas wo value ignore ho jati hai (jaise method hai hi nahi). Purane zamane me JS me achha "error" concept nahi tha. Iske ulta, `Symbol.toPrimitive` **zyada strict hai**, ye primitive **return karna hi padega**, warna error aayega.

5. Aage ke conversions

Kai operators aur functions type conversions karte hain, jaise `*` operands ko numbers me badalta hai.

Agar argument me object do to **do stages** hote hain:

1. Object **primitive me convert** hota hai (upar ke rules se).
2. Zarurat ho to us primitive ko **aur convert** kiya jata hai.

javascript

```
let obj = {
  // baaki methods na ho to toString saare conversions handle karta hai
  toString() {
    return "2";
  }
};

alert(obj * 2); // 4, object primitive "2" bana, phir multiplication ne use num
```

1. `obj * 2` pehle object ko primitive (string `"2"`) me badalta hai.
2. Phir `"2" * 2` = `2 * 2` (string number me convert hoti hai).

Binary plus isi situation me strings jod deta hai, kyunki wo string khushi se le leta hai:

javascript

```
let obj = {
  toString() {
    return "2";
  }
};

alert(obj + 2); // "22" ("2" + 2), primitive me conversion se string mili => co
```

Summary

Object-to-primitive conversion **kai built-in functions aur operators** dwara automatically call hota hai jo primitive value chahte hain.

Iske 3 types (hints):

- `"string"` (`alert` aur string chahne wale operations ke liye)
- `"number"` (maths ke liye)
- `"default"` (kuch hi operators, aur objects aam taur par ise `"number"` jaisa implement karte hain)

Specification me explicitly likha hai ki kaunsa operator kaunsa hint use karta hai.

Conversion algorithm:

1. `obj[Symbol.toPrimitive](hint)` call karo agar method hai.
2. Nahi to hint `"string"` ho to `obj.toString()` ya `obj.valueOf()` try karo.
3. Nahi to hint `"number"` ya `"default"` ho to `obj.valueOf()` ya `obj.toString()` try karo.

Ye saare methods (agar defined hain) **primitive return karne chahiye**.

Practice me aksar sirf `obj.toString()` implement karna kaafi hota hai, jo object ka **human-readable** representation (logging ya debugging ke liye) return kare, sab string conversions ke liye "catch-all" ki tarah.

Quick Summary

Baat	Yaad rakho
Boolean conversion	Objects hamesha <code>true</code>
<code>"string"</code> hint	<code>alert(obj)</code> , <code>obj</code> ko key banana
<code>"number"</code> hint	Maths, <code>+obj</code> , <code><</code> , <code>></code>
<code>"default"</code> hint	Binary <code>+</code> , <code>obj == 1</code>
Pehli priority	<code>obj[Symbol.toPrimitive](hint)</code>
String hint par	<code>toString</code> → <code>valueOf</code>
Number/default hint par	<code>valueOf</code> → <code>toString</code>
Plain object default	<code>toString()</code> = <code>"[object Object]"</code>
Kya return karein	Primitive (object nahi)
Practical tip	Sirf <code>toString()</code> implement karna aksar kaafi hai
Result	<code>obj1 + obj2</code> ka result kabhi object nahi hota